The Journal of Open Source Software

DeepAudioX: A PyTorch-Based Library for Audio Learning with Pretrained Self-Supervised Audio Backbones

Christos Nikou [1¶], **Stefanos Vlachos** [1], **Ellie Vakalaki** [1], **and Theodoros Giannakopoulos** [1]

1 Multimedia Analysis Group of the Computational Intelligence Laboratory (MagCIL), Institute of Informatics and Telecommunications, NCSR DEMOKRITOS ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 08 March 2026
Published: unpublished

Summary

DeepAudioX is an open-source Python library built on PyTorch that provides simple and flexible pipelines for audio classification using pretrained audio foundation models as feature extractors. The library reduces boilerplate code while preserving extensibility, enabling researchers, students, and practitioners to rapidly prototype and deploy audio classification systems. It is easily customizable, allowing users to integrate their own architectures while leveraging the rest of the framework.

Statement of need

Robust audio foundation models have become an integral component of research in a wide variety of audio tasks. However, their use comes with a significant drawback: as each model is available in different repositories, they usually have different requirements regarding data structures and handling, so that model-specific preprocessing and code adaptation becomes unavoidable. As a result, users have to invest significant amounts of time understanding and integrating each model separately. This issue becomes particularly pronounced in benchmarking scenarios, where multiple models are evaluated and compared on the same task. `DeepAudioX` has been developed to address this problem by providing a user-friendly interface to enable the consistent integration of various audio foundation models. The library has been designed to be extensible, allowing the integration of additional models in the future. It provides a consistent way for representation extraction, along with a pipeline for downstream tasks, while supporting different pooling strategies and customizable classification heads. By simplifying the integration of different models and by offering a standardized evaluation workflow, `DeepAudioX` minimizes development time and paves the way towards a more uniform and reproducible research process in audio machine learning.

State of the field

The Python audio ecosystem spans a wide range of libraries that address digital signal processing, music information retrieval, and deep learning–based modeling. Classical audio analysis toolkits such as `Essentia` (Bogdanov et al., 2013), `pyAudioAnalysis` (Giannakopoulos, 2015), and `librosa` (McFee et al., 2015) provide extensive functionality for feature extraction, spectral analysis, and statistical descriptors, and as such, they are widely used for exploratory analysis and traditional machine-learning pipelines. While these libraries are highly effective for low-level signal processing and handcrafted feature engineering, they do not natively offer streamlined

end-to-end workflows centered on modern pretrained neural audio backbones. With the rise of deep learning, frameworks such as `PyTorch` (Paszke et al., 2019) and `torchaudio` (Yang et al., 2022) have become foundational tools for building custom neural audio systems, providing efficient tensor operations, data loading, and core audio transforms. Complementing these building blocks, the `Hugging Face Transformers` ecosystem (Wolf et al., 2019) has popularized the use of large pretrained foundation models, enabling researchers to leverage state-of-the-art encoders across modalities, including audio. However, these frameworks primarily operate at a low or mid level of abstraction, often requiring significant engineering effort to assemble full training, evaluation, and deployment pipelines for specific downstream tasks. Higher-level research toolkits such as `SpeechBrain` (Ravanelli et al., n.d.) extend these capabilities by offering modular recipes and broad task coverage across speech and audio domains, including speech recognition, speaker identification, and enhancement. These platforms emphasize flexibility and research experimentation, but their breadth and configurability can introduce complexity for users seeking rapid development of narrowly scoped applications. In parallel, model-centric efforts such as `PANNs` (Kong et al., 2020) have demonstrated the effectiveness of large-scale pretrained audio neural networks, primarily serving as embedding extractors or baseline architectures rather than complete application pipelines. Within this landscape, existing tools either prioritize low-level signal analysis, generic deep-learning infrastructure, or broad research-oriented frameworks. There remains a need for lightweight, task-focused libraries that bridge pretrained audio foundation models with concise, production-oriented classification workflows, reducing boilerplate while preserving extensibility for applied practitioners.

Software Design

The optimal goal of `DeepAudioX` is to serve as a higher-level abstraction layer and provide a rich set of well-documented and reusable classes and functions that can streamline tedious operations typically encountered in audio classification tasks, such as processing/loading audio datasets, building complex model architectures, and designing efficient training loops.

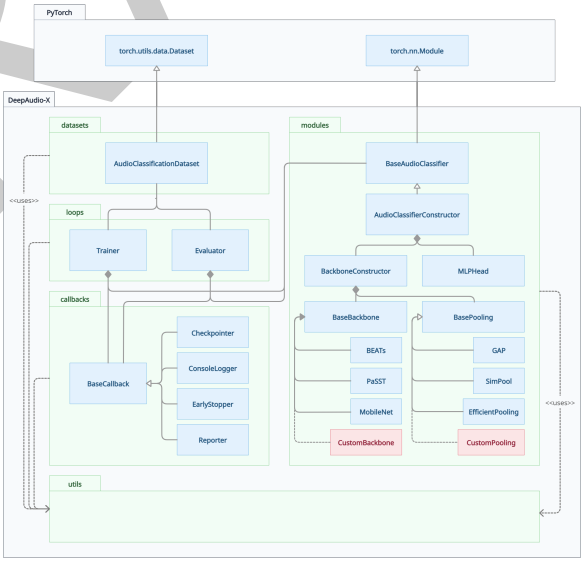

Figure 1: Class diagram of the `DeepAudioX` library, illustrating the five core packages (datasets, modules, loops, callbacks, and utils) and their relationships with PyTorch base classes.

In order to meet these requirements, the `DeepAudioX` codebase adopts a wide set of object-oriented design patterns, fostering modularity, re-usability and extensibility. The library is

fully PyTorch-native, allowing seamless integration with existing training tools and enabling more sophisticated users to override or extend internal components without modifying the core framework. The class diagram of Figure 1 provides a deeper look into the composition of `DeepAudioX`, and outlines five core packages: (i) datasets, (ii) modules, (iii) loops, (iv) callbacks, and (v) utils.

datasets The `datasets` package provides access to standardized PyTorch dataset classes, designed to streamline data ingestion and pre-processing across a variety of audio-specific tasks. The current distribution of `DeepAudioX` provides a single prototype dataset, `AudioClassificationDataset`, suited for automatically loading audio files from the local file system, applying segmentation and organizing data for the training loop. The adoption of the Factory Method design pattern allows for effortlessly loading data either given a local directory or a Python dictionary (e.g. metadata loaded from a JSON file).

modules The `modules` package provides all the essential building blocks for constructing end-to-end audio classifiers.

Backbones: Starting from the bottom of the module hierarchy, `DeepAudioX` makes available a powerful set of state-of-the-art (SoTA) transformer and CNN-based networks. Ranging from lightweight to heavier architectures, these networks can serve as audio encoders/model backbones (inheriting from `BaseBackbone`) and even transfer the knowledge from pre-trained model weights, reducing training time and boosting classification performance. To this date, `DeepAudioX` incorporates BEATs (Chen et al., 2022), PaSST (Koutini et al., 2022) and MobileNet (Howard et al., 2017), however, the modular architecture of the library allows for future expansion of this list. The weights of these models are stored in a separate public repository and downloaded upon initialization of the respective modules. This keeps the library lightweight, giving users full control over downloading additional files.

Pooling: Motivated by the ability of attention-based pooling methods to produce feature-map aggregations of higher quality compared to conventional global average pooling, `DeepAudioX` features two attention-based pooling modules, `SimPool` (SP) (Psomas et al., 2023) and `Efficient Probing` (EP) (Psomas et al., 2025), in addition to a Global Average Pooling (GAP) module (all inheriting from `BasePooling`), which can be integrated on any backbone.

Assembly: A fully customizable `MLPHead` can be appended at the top of the module sequence, resulting in an end-to-end audio classifier. Taking advantage of the Registry design pattern, all aforementioned components can be easily instantiated and used as distinct modules, given their corresponding names. Nevertheless, the true strength of `DeepAudioX` lies in `AudioClassifierConstructor`. By accepting either string identifiers or direct dependency injection of backbone and pooling objects, this factory class facilitates the seamless assembly of complete audio classifiers, abstracting the complexities of model creation into a few lines of code.

loops The `loops` package can be viewed as the engine of `DeepAudioX`, since it handles the execution of training and validation pipelines. Leveraging class composition, `Trainer` and `Evaluator` are built via injection of all the components typically required throughout the training lifecycle (data loaders, model, optimizer, loss function, optionally learning rate scheduler) and shield users from intricate training logic, such as iterating over data batches, managing forwarding and backpropagation, aggregating loss measurements, and saving checkpoints.

callbacks The `callbacks` package allows for the decoupling of auxiliary tasks from the main execution flow of training and evaluation loops. The `Trainer` and `Evaluator` are designed to hold a list of callback objects, which are configured to perform certain actions throughout the training lifecycle. Specifically, `Checkpointer` is responsible for monitoring the validation loss at the end of each epoch and persisting model weights when a better performance is achieved, `EarlyStopper` also acts at the end of each epoch tracking stagnation and terminating training after a defined patience threshold, `ConsoleLogger` renders real-time messages to the user, and `Reporter` generates a classification report upon completion of the testing phase.

utils The `utils` package contains a suite of reusable auxiliary functions that support the core modules of the library.

Research and Community Impact

`DeepAudioX` contributes to the research community as it can drastically minimize the time required for integrating and benchmarking audio foundation models. By standardizing these models' use and evaluation, the package allows researchers to focus on experimental design and analysis instead of time-consuming code adaptations for each model. `DeepAudioX` not only makes benchmarking easier, as users are enabled to compare various models' performance using a unified interface, but also promotes fair comparison and transparent reporting of results across studies with the use of the exact same testing process. Furthermore, easily customizable classification heads that are suitable for each task are offered, requiring no additional code to implement, only specifying a few parameters. The package is open source and user-friendly, giving researchers the opportunity to work more efficiently and enabling non-experts to utilize these models. Students can also benefit from exploring audio model development and experimentation, without the need of extensive coding. Finally, it establishes a common way of handling and testing different powerful and widely used architectures, facilitating consistent performance comparisons while minimizing the time needed for prototyping and benchmarking.

Case Study

We evaluate `DeepAudioX` on three audio classification benchmarks — keyword spotting, speech emotion recognition, and environmental sound classification — reporting accuracy across all backbone and pooling combinations.

Datasets We use the following three benchmark datasets: *SpeechCommands 5h* (Warden, 2018), *CREMA-D* (Cao et al., 2014) and *ESC-50* (Piczak, 2015). SpeechCommands is a collection of audio recordings of spoken words designed for training and benchmarking keyword spotting systems. CREMA-D is an audio-visual dataset for emotion recognition consisting of facial and vocal emotional expressions in spoken sentences. Finally, ESC-50 is a labeled collection of environmental audio recordings suitable for benchmarking in environmental sound classification. The splits used for training and testing are those proposed by the HEAR benchmark (Turian et al., 2022). Table 1 provides an overview of the characteristics of each dataset.

Table 1: Dataset characteristics

Dataset	# Clips	# Classes	Duration (sec)	Evaluation Metric
SpeechCommands 5h	22,890	36	1.0	Accuracy
CREMA-D	7,438	6	5.0	Accuracy
ESC-50	2,000	50	5.0	Accuracy

Models Each model is the combination of the chosen backbone (*BEATs*, *PaSST* and *MobileNet (MN)*) to serve as feature extractor and the pooling strategy (*GAP*, *SP*, and *EP*) to map the feature embeddings into a 1-Dimensional feature vector. We utilize two variants of MobileNets, i.e., the *MobileNet-05* (width multiplier = 0.5), and the *Mobilenet-10* (width multiplier =1.0) as presented in (Schmid et al., 2023). Therefore, a total of 12 models are assembled, allowing to evaluate the performance of the library across varying architecture sizes. A simple linear layer is appended on top of each model using the *MLPHead* class to map the feature embedding to the total number of classes.

Experimental Setup During training, pre-trained *BEATs* and *PaSST* backbones are kept frozen. On the other hand, considering the significantly lighter architecture of MobileNets, the

backbone weights are fine-tuned on the new tasks. Regarding training hyper-parameters, data are sampled in batches of 256 and models are trained for a maximum of 200 *epochs* (with a *patience* of 15 epochs). Model parameters are optimized using the *Adam* optimizer (Kingma & Ba, 2014) starting with a learning rate of 10^{-3}. Learning rate scheduling is performed using the *ReduceLROnPlateau* PyTorch scheduler. The objective function is the cross entropy loss. All these parameters correspond to the default parameters of the *Trainer* class.

Results Classification results reported in Table 2 demonstrate the capacity of DeepAudioX to produce powerful audio classifiers across a wide range of audio-related tasks (Keyword Spotting, Speech Emotion Recognition, Sound Event Classification), minimizing coding overhead and streamlining the creation of effective audio training pipelines. In the table, SR denotes the input sample rate (in Hz) expected by each backbone, and Frz. indicates whether the backbone weights were frozen during training (T = frozen, F = fine-tuned).

Table 2: Classification accuracy of all backbone and pooling strategy combinations across three benchmark datasets.

Backbone	Pool.	SR	Frz.	SpeechCmds	CREMA-D	ESC-50
BEATs	GAP	16k	T	0.61	0.6166	0.9850
BEATs	SP	16k	T	0.78	0.6589	0.9835
BEATs	EP	16k	T	0.83	0.6461	0.9810
PaSST	GAP	32k	T	0.46	0.5925	0.9805
PaSST	SP	32k	T	0.69	0.7069	0.9765
PaSST	EP	32k	T	0.88	0.6963	0.9785
MN-05	GAP	32k	F	0.93	0.7851	0.9415
MN-05	SP	32k	F	0.94	0.7807	0.9575
MN-05	EP	32k	F	0.93	0.8031	0.9445
MN-10	GAP	32k	F	0.94	0.7737	0.9750
MN-10	SP	32k	F	0.91	0.7547	0.9680
MN-10	EP	32k	F	0.94	0.8030	0.9695

AI Usage Disclosure

Generative AI tools were used in the development of this work in a limited and auxiliary capacity. Specifically, AI-assisted coding tools (Claude Sonnet 4.6) were used to support tasks such as code formatting and test generation. Conversational AI tools (Claude Sonnet 4.6 & ChatGPT-5.3) were also consulted for discussions on coding efficiency and implementation strategies. The core intellectual contributions of this work — including the conceptual design of the library, its architectural structure, workflow design, and abstract class definitions — represent the authors' original intellectual work. With respect to the manuscript, the AI tools were used solely for grammar checking and language refinement, with all scientific content authored by the authors.

References

Bogdanov, D., Wack, N., Gómez Gutiérrez, E., Gulati, S., Herrera Boyer, P., Mayor, O., Roma Trepat, G., Salamon, J., Zapata González, J. R., & Serra, X. (2013). Essentia: An audio analysis library for music information retrieval. *Britto a, Gouyon f, Dixon s, Editors. 14th Conference of the International Society for Music Information Retrieval (ISMIR); 2013 Nov 4-8; Curitiba, Brazil.[place Unknown]: ISMIR; 2013. P. 493-8.*

Cao, H., Cooper, D. G., Keutmann, M. K., Gur, R. C., Nenkova, A., & Verma, R. (2014). CREMA-D: Crowd-sourced emotional multimodal actors dataset. *IEEE Transactions on Affective Computing*, *5*(4), 377–390.

Chen, S., Wu, Y., Wang, C., Liu, S., Tompkins, D., Chen, Z., & Wei, F. (2022). Beats: Audio pre-training with acoustic tokenizers. *arXiv Preprint arXiv:2212.09058*.

Giannakopoulos, T. (2015). Pyaudioanalysis: An open-source python library for audio signal analysis. *PloS One*, *10*(12), e0144610.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *CoRR*, *abs/1704.04861*. http://arxiv.org/abs/1704.04861

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv Preprint arXiv:1412.6980*. https://doi.org/10.48550/arxiv.1412.6980

Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W., & Plumbley, M. D. (2020). Panns: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, *28*, 2880–2894.

Koutini, K., Eghbal-zadeh, H., Seeger, M., & Widmer, G. (2022). PaSST: Efficient training of audio transformers with patchout. *Interspeech*.

McFee, B., Raffel, C., Liang, D., Ellis, D. P., McVicar, M., Battenberg, E., Nieto, O., & others. (2015). Librosa: Audio and music signal analysis in python. *SciPy*, *2015*(18-24), 7.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*.

Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. *Proceedings of the 23rd Annual ACM Conference on Multimedia*, 1015–1018. https://doi.org/10.1145/2733373.2806390

Psomas, B., Christopoulos, D., Baltzi, E., Kakogeorgiou, I., Aravanis, T., Komodakis, N., Karantzalos, K., Avrithis, Y., & Tolias, G. (2025). Attention, please! Revisiting attentive probing for masked image modeling. *Greeks in AI Symposium 2025*.

Psomas, B., Kakogeorgiou, I., Karantzalos, K., & Avrithis, Y. (2023). Keep it simpool: Who said supervised transformers suffer from attention deficit? *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 5350–5360.

Ravanelli, M., Parcollet, T., Plantinga, P., Rouhe, A., Cornell, S., Lugosch, L., Subakan, C., Dawalatabad, N., Heba, A., Zhong, J., Chou, J.-C., Yeh, S.-L., Fu, S.-W., Liao, C.-F., Rastorgueva, E., Grondin, F., Aris, W., Na, H., Gao, Y., … Bengio, Y. (n.d.). *SpeechBrain*. https://github.com/speechbrain/speechbrain/

Schmid, F., Koutini, K., & Widmer, G. (2023). Efficient large-scale audio tagging via transformer-to-cnn knowledge distillation. *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 1–5.

Turian, J., Shier, J., Khan, H. R., Raj, B., Schuller, B. W., Steinmetz, C. J., Malloy, C., Tzanetakis, G., Velarde, G., McNally, K., & others. (2022). HEAR: Holistic evaluation of audio representations. *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, *176*, 125–145. https://proceedings.mlr.press/v176/turian22a.html

Warden, P. (2018). Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. *ArXiv e-Prints*. https://arxiv.org/abs/1804.03209

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & others. (2019). Huggingface's transformers: State-of-the-art natural language processing. *arXiv Preprint arXiv:1910.03771*.

Yang, Y.-Y., Hira, M., Ni, Z., Astafurov, A., Chen, C., Puhrsch, C., Pollack, D., Genzel, D., Greenberg, D., Yang, E. Z., & others. (2022). Torchaudio: Building blocks for audio and

speech processing. *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 6982–6986.